

Aula 5 - Funções

Essa aula se resume ao estudo da criação de funções em C++, bem como funções recursivas.

O que são?

Do início do curso até hoje, vocês já aprenderam várias coisinhas úteis de C++ que fazem coisas interessantes:

- `cout` imprime algo na tela
- `max(x, y)` retorna o maior valor entre x e y
- `.push_back(x)` específica dos vectors, enfia um valor no final de um vector

E por aí vai.

Cada uma dessas partículas de linguagem que fazem coisas legais é chamada de **função**. Uma **biblioteca** em C++ nada mais é do que um **conjunto de funções** pré-definidas:

```
#include <iostream> // cin, cout
#include <iomanip> // fixed, setprecision
#include <vector> // vectors, push_back, size
...
```

Hoje, vocês vão aprender a criar suas próprias funções.

Criando e declarando funções

As funções na programação se assemelham muito às funções da matemática. Na matemática, uma função recebe uma entrada e devolve uma saída baseada naquela entrada.

$f(x) = x^2$ nos dará o número 4 quando fornecermos o número 2 de entrada, por exemplo.

Em programação, seguimos o seguinte formato para declarar funções:

```
<tipo_de_retorno> <nome_da_função>(<parametros_de_entrada>{
    // tudo que ela faz
```

```
    return <saída>;  
}
```

Sendo:

- **<tipo_de_retorno>** o tipo da informação que essa função retorna. No caso da $f(x)$ que eu dei de exemplo antes, ela retorna um número real (float)
- **<nome_da_função>** é o nome que você escolher para ela
- **<parametros_de_entrada>** é o que vai ser passado para a função (como o número x da $f(x)$)
- **<saída>** é o que será devolvido pela função (como o valor de x^2 no nosso exemplo)

Traduzindo a função $f(x)$ para C++, ficaria:

```
float quadrado(float x){  
    return x * x;  
}
```

Uma função que recebe um número real, devolve um número real, e se chama "quadrado".

Escopo de variáveis

Alguns de vocês já se depararam com esse problema anteriormente em alguns exercícios. Uma vez que uma variável é declarada, ela só existe dentro do "contexto" na qual ela foi declarada (normalmente delimitado por chaves { }).

```
for(int i = 0; i<n; ++i){  
    // ...  
    int m;  
}  
cout << i; // DÁ ERRO!  
cout << m; // TAMBÉM DÁ ERRO!!!  
// As duas só existiam dentro do for  
  
if(2 > 3){  
    int a = 2;
```

```
}
```

```
a = 3; // ERR00000!!!
```

Mas, se uma variável é declarada num escopo mais externo, ela pode ser usada tranquilamente:

```
int a;  
if(2 + 2 = 4){  
    a = 3; // funciona!  
}
```

Quando dizemos que uma variável está declarada no **escopo global**, ela está fora da função `main`:

```
#include <bits/stdc++.h>  
using namespace std;  
  
int x;  
  
int main(){  
    // x existe aqui  
}
```

E isso vale pras funções também. Então, por convenção (e para que sua função exista em todo o código) declarem suas funções fora da main!

```
#include <bits/stdc++.h>  
using namespace std;  
  
float quadrado(float x){  
    return x*x;  
}  
  
int main(){  
    // e agora podemos usar a quadrado aqui dentro  
    int n = 2;
```

```
    cout << quadrado(n);  
}
```

Passagem de parâmetros

Ao declarar funções, temos dois tipos de passagem de parâmetro (a forma como as variáveis são passadas para as funções):

Passagem por valor

A forma como escrevemos a `quadrado` funciona com passagem por valor, isso significa que, ao ser executada, a função cria uma **cópia** do valor da variável que ela foi chamada. No caso, a `quadrado` quando chamada na variável `n` cria uma cópia dela, e trabalha em cima dessa cópia. Isso significa que:

```
int n = 2;  
quadrado(n);  
cout << n; // vai imprimir 2  
cout << quadrado(n); // vai imprimir 4
```

Ou seja, na passagem por valor, a função **NÃO ALTERA** a variável em si, mas sim mexe com o valor que ela contém.

Passagem por referência

Se quisermos que a função altere de fato a variável, precisamos modificá-la da seguinte forma:

```
float quadrado(float& x){  
    return x*x;  
}
```

O `&` diz para o compilador que essa função funciona com passagem de variáveis por referência, isso significa que o próprio endereço de memória da variável `x` é passado para a função, e, portanto, o que está contido nela é modificado.

```
int n = 2;  
quadrado(n);
```

```
cout << n; // vai imprimir 4 com a nova modificação
```

Recursão

Na matemática, existem funções como a de Fibonacci, em que temos

$$fib(n) = fib(n - 1) + fib(n - 2) \qquad fib(2) = fib(1) = 1$$

Isso é uma função que é definida a partir dela mesma, aplicada a números anteriores, com a especificação de um caso base (um caso de parada). Essa é exatamente a característica que define uma função **recursiva**.

Outro bom exemplo é a função fatorial:

$$fat(n) = n \times fat(n - 1) \qquad fat(1) = 1$$

Também é definida a partir de casos anteriores, com um caso base.

A função de Fibonacci é dita de "grau 2", dado que ela é definida a partir de dois termos anteriores, enquanto a fatorial é dita de "grau 1", dado que ela é definida por só um termo anterior. Note que a quantidade de casos base varia de acordo com o grau da função. Grau X implica X casos base.

Em C++, temos algo muito parecido com a matemática:

```
int fib(int n){
    if(n == 1 || n == 2) return 1;

    return fib(n-1) + fib(n-2)
}

int fat(int n){
    if(n == 1) return 1;

    return n * fat(n-1);
}
```

Notem que eu não usei cláusulas `else` nessas funções. Isso é porque, ao executar um `return`, a função se encerra automaticamente, as outras linhas não são nem lidas pelo compilador (é como um `break` para funções).